
Part B

Ex: 7

Title: Optimal Road Network

Problem Description: A local government wants to improve the road network between towns to enhance connectivity and reduce travel costs. The region consists of five towns that need to be connected by the most efficient road network possible, minimizing the total length of the roads while ensuring that each town is accessible from any other town.

Method: A greedy Technique for a spanning tree which has a total minimum weight. Use Union-Find Data Structure to detect cycles during the execution of the algorithm.

Theory Reference: Module 3

Algorithm

1. Initialize Parent Array:

- Create an array `par[]` of size `n` and initialize all elements to `-1`, representing that each node is initially its own set.

2. Initialize Variables:

- `mincost` to store the total cost of the Minimum Spanning Tree (MST).
- `ne` (number of edges in MST), initially `0`.
- `a`, `b`, `u`, `v` to store the nodes of the minimum edge.

3. Repeat Until the MST Contains `n-1` Edges:

- Find the minimum cost edge in `cost[][]`.
- Store the indices `a` and `b` of the minimum edge.
- Store `u` and `v` (representative nodes of `a` and `b` using the Find operation).

4. Find the Parent (Find Operation):

- Use path traversal to find the representative (root) of `u`.
- Use path traversal to find the representative (root) of `v`.

5. Check for Cyclicity:

- If `u` and `v` are different, it means adding this edge won't form a cycle.
 - Print the edge and its cost.
 - Update the total minimum cost `mincost`.
-

- Perform the **Union Operation**: Merge the sets by updating $par[v] = u$ (or vice versa).
- Increment ne (edge count in MST).

6. Mark the Edge as Processed:

- Set $cost[a][b] = cost[b][a] = Integer.MAX_VALUE$ to avoid selecting it again.

7. Output the Total Cost of the MST.

Program

```
import java.util.Scanner;

public class Kruskal {

    static void kruskal(int cost[][[]],int n)

    {

        int[] par = new int[n];

        for(int i=0;i<n;i++)

            par[i]=-1;

        int a=0,b=0,u=0,v=0,min, mincost = 0, ne = 0;

        while(ne < n-1)

        {

            //Find the least cost edge

            min = Integer.MAX_VALUE;

            for(int i=0;i<n;i++)

            {

                for(int j=0;j<n;j++)

                {
```

```
        if(cost[i][j] < min)
        {
            min=cost[i][j];
            a=u=i;
            b=v=j;
        }
    }
}

//Find - find the representative of a and b as u and v respectively

while(par[u]!=-1)
    u=par[u];
while(par[v]!=-1)
    v=par[v];

// Union - Check if edge select cause cyclicity?

// (if u==v) means reject that edge, as cycle is detected

//if(u!=v) means no cycle, so apply union operation)

if(u!=v)
{
    System.out.println("Cost of Laying road from town "+ a +" to town " + b +" is "+
min +" Lakhs");

    mincost+=min;

    // Make the representative of v as u

    if(u<v)
```

```
        par[v]=u;

        // Make the representative of u as v

    else

        par[u]=v;

        ne++;

    }

    //edge included in MST should not be considered for next iteration

    cost[a][b]=cost[b][a]=Integer.MAX_VALUE;

}

System.out.println("Cost of Laying entire Road Network =" + mincost + " Lakhs");

}

public static void main(String[] args)

{

    int n;

    Scanner s = new Scanner(System.in);

    System.out.println("Enter the no. of towns");

    n = s.nextInt();

    int[][] c = new int[n][n];

    System.out.println("Enter the cost of laying roads between two towns in lakhs");

    for(int i=0;i<n;i++)

    {

        for(int j=0;j<n;j++)
```

```

{
    c[i][j]=s.nextInt();

    if (c[i][j] == 0)

        c[i][j] = Integer.MAX_VALUE;

}

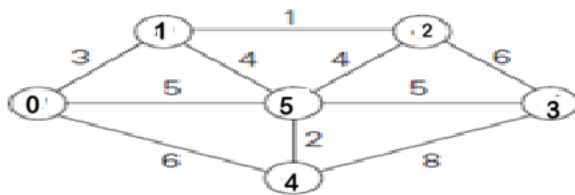
}

kruskal(c,n);

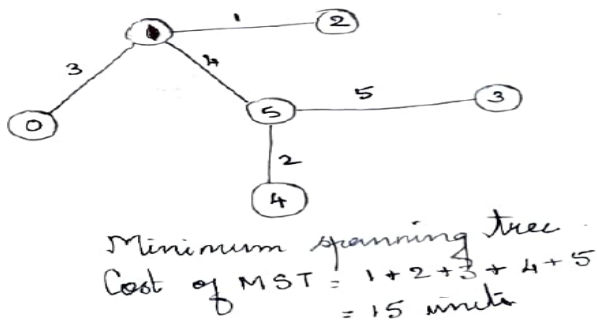
}

}

```

Output

Edges arranged in ascending order of weights: 1-2, 4-5, 0-1, 1-5, 2-5, 0-5, 3-5, 0-4, 2-3, 3-4.



```
run:
Enter the no. of towns
6
Enter the cost of laying roads between two towns in lakhs
0 3 0 0 6 5
3 0 1 0 0 4
0 1 0 6 0 7
0 0 6 0 8 5
6 0 0 8 0 2
5 4 4 5 2 0
Cost of Laying road from town 1 to town 2 is 1 Lakhs
Cost of Laying road from town 4 to town 5 is 2 Lakhs
Cost of Laying road from town 0 to town 1 is 3 Lakhs
Cost of Laying road from town 1 to town 5 is 4 Lakhs
Cost of Laying road from town 3 to town 5 is 5 Lakhs
Cost of Laying entire Road Network =15 Lakhs
BUILD SUCCESSFUL (total time: 8 seconds)
```
